

Avancee en apprentissage statistique

Andres Espinosa

September 14, 2026

Contents

1	Monday September 7th	1
1.1	Linear Regression	1
1.2	Bayesian LR	3
1.3	Predictions	3
1.3.1	Predictive Distribution	3
2	Monday September 14th	3
2.1	Gaussian Process Regression	3
2.1.1	Gaussian Process Regression	4
2.1.2	Kernel Function	5
2.1.3	Function-Space View	5

1 Monday September 7th

I arrived to this class and also missed last week's class. They talked about parameter estimation and maximum likelihood, it seems. We are now moving onto multiple linear regression.

1.1 Linear Regression

Simple linear regression is a special case of multiple linear regression. Consider the case in which we have a scalar real-valued output variable $y \in \mathbb{R}$ and a vector of input variables $\mathbf{x} \in \mathbb{R}^n$. In linear regression, we have a vector of weights $\mathbf{w} \in \mathbb{R}^n$ and a zero-mean gaussian random variable $\varepsilon \sim N(0, P^2)$. We declare $f(x)$, a function that relates our weights to a predicted value of y .

$$f(x) = \mathbf{w}^\top \mathbf{x} \quad (1)$$

with this function, we can measure the difference between our predictions and our real-value and attribute it to the error ε .

$$y = f(x) + \varepsilon \quad (2)$$

$$\hat{y} \approx f(x) \quad (3)$$

Naturally, this leads to the question of how we determine our weight vector $\mathbf{w}$ that we use to get our linear regression model. If we have a data sample: $\{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$, we can use that to get our best guess for those values of the weight vector. We can use maximum likelihood estimate to estimate the values.

$$L(w|data) = \mathbb{P}(data|w) = \prod_{i=1}^m g_y(y_i|x_i, w) \quad (4)$$

$$= \prod_{i=1}^m (2\pi)^{-1/2} (P^2)^{-1/2} e^{-\frac{1}{2P^2} (y_i - \mathbf{w}^\top \mathbf{x})^2} \quad (5)$$

$$= \prod_{i=1}^m (2\pi)^{-1/2} (P^2)^{-1/2} e^{-\frac{1}{2P^2} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2} \quad \text{matrix case} \quad (6)$$

We can stack our data samples next to each other in vectors to facilitate this maximum likelihood.

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}, \mathbf{X} = \begin{bmatrix} \mathbf{x}_1^\top \\ \mathbf{x}_2^\top \\ \vdots \\ \mathbf{x}_m^\top \end{bmatrix}, \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_n \end{bmatrix} \quad (7)$$

So we have $\mathbf{y} \in \mathbb{R}^m, \mathbf{X} \in \mathbb{R}^{m \times n}, \mathbf{w} \in \mathbb{R}^n$. With maximum likelihood estimation, we can solve the below least-squares equation to get our predicted $\hat{\mathbf{w}}$.

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} = \mathbf{X}^\dagger \mathbf{y} \quad (8)$$

We can then get our predicted values for y , based off of any x using the model $f(x) = \mathbf{w}^\top \mathbf{x}$. If we have a series of inputs $\mathbf{X}$, we can turn it into a series of predictions with

$$\hat{\mathbf{y}} = \mathbf{X} \hat{\mathbf{w}} \quad (9)$$

What is the distribution of $\hat{\mathbf{w}}$?

$$\hat{\mathbf{w}} \sim N(w, P^2 (\mathbf{X}^\top \mathbf{X})^{-1}) \quad (10)$$

We will prove this in the practical session.

We can use the maximum likelihood estimate to estimate our value for P

$$\hat{P}_{ML}^2 = \frac{1}{m} \sum_{i=1}^m \hat{\varepsilon}_i^2 \quad (11)$$

where $\hat{\varepsilon} = y_i - \hat{y}_i$. It is an estimate because we are using the predicted weights $\hat{\mathbf{w}}$. We then remove the bias by doing the empirical

$$\hat{P}^2 = \frac{1}{m - (n - 1)} \sum_{i=1}^m \hat{\varepsilon}_i^2 \quad (12)$$

Remark: We might have problems when computing $(\mathbf{X}^\top \mathbf{X})^{-1}$. If we have repeated data points, then the matrix is not full rank and it is therefore not invertible. We can add a tiny noise onto the diagonal of the matrix so that we ensure full rank, and that we can compute the inverse.

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X} + \xi I_p)^{-1} \mathbf{X}^\top \mathbf{y} \quad (13)$$

where ξ is a tiny noise with arbitrary distribution.

1.2 Bayesian LR

Here we move onto a bayesian form of linear regression. I do not fully understand what that is at the moment.

We set a prior on $w \sim N(0, \Sigma_n)$, that it follows a normal distribution with zero mean and covariance Σ_p .

$$P(\mathbf{w}|\mathbf{X}, \mathbf{y}) = \frac{P_y(y|\mathbf{X}, \mathbf{w})P(\mathbf{w})}{P_y(\mathbf{y}|\mathbf{X})} \quad (14)$$

$$P(\mathbf{w}|\mathbf{X}, \mathbf{y}) \propto e^{-\frac{1}{2}((\mathbf{w}-\bar{\mathbf{w}})^\top \mathbf{A}(\mathbf{w}-\bar{\mathbf{w}}))} \quad (15)$$

where

$$\bar{\mathbf{w}} = \sigma^{-2} \mathbf{A} \mathbf{X}^\top \mathbf{y}, \quad \mathbf{A} = \sigma^{-2} \mathbf{X}^\top \mathbf{X} + \Sigma_n^{-1} \quad (16)$$

If we don't put any weight on our covariance matrix Σ , then we end up getting the same maximum likelihood estimation as the $\hat{\mathbf{w}}$ ends up being the same as previously. If we have it be the ξI_n , then it is the same as the previous with that tiny regularization that allows us to compute the inverse. We can choose whatever we want (I am still unclear on how or why we would do that) for Σ_n , as long as it is a covariance matrix—symmetric ($\Sigma = \Sigma^\top$) and PSD ($\Sigma \succeq 0$).

1.3 Predictions

Why would we want other covariances? It seems that it enables us to regularize or penalize the model by putting assumptions on the covariance. It allows us as the model designer to have some say in how good we think our model is, and then embed those assumptions in the model to improve them accordingly. Maximum Likelihood goes to Maximum A Posteriori.

$$\hat{y}_{MAP} = \mathbf{x}^\top \hat{\mathbf{w}}_{MAP} \quad (17)$$

This allows us to have a posterior $\mathbf{w} \sim N(\bar{\mathbf{w}}, \mathbf{A}^{-1})$. It helps us get a confidence on our outputs because the covariance matrix gives us the full predictive distribution. I can't pretend I fully understand what that means yet but I am much closer.

1.3.1 Predictive Distribution

We now have the same training data set $\mathbf{X} \in \mathbb{R}^{m \times n}, \mathbf{y} \in \mathbb{R}^m$. Now, we get new inputs $\mathbf{X}^*$, and we want to not only have an output but we also want to have a confidence of our prediction based off of the inputs and the output we get f^* .

Our predictive maximum a posteriori distribution propagates into the distribution of our vector of new outputs. So we can model a portion of our uncertainty that comes from our weights.

2 Monday September 14th

2.1 Gaussian Process Regression

If we have a nonlinear process that we would like to model linearly, then we can augment the variable vector to capture the nonlinearity. For example,

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_n x_1^2 \\ \vdots \\ x_1 x_2 \dots x_{n-1} x_n^2 \\ x_n \end{bmatrix} \quad (18)$$

There are many different ways that the augmented vector can be created. The log function, or just the square, or square root, etc. This augmentation process then becomes a part of the problem for the modeler, as they must use some information that they have on the real-world model and embed that understanding in the function choices. Poor choices of augmentation can then increase the amount of variables significantly without adding that much valuable information from it. With more variables, we also increase our chance of overfitting on our data, if our data is not sufficiently diverse enough.

We can formalize this process. We have an input vector $\mathbf{x} \in \mathbb{R}^n$, and a nonlinear transformation of $\mathbf{x}$, $\phi(\mathbf{x}) \in \mathbb{R}^p$. $\{\phi : \mathbf{x} \rightarrow \phi(\mathbf{x})\}$ is a function that maps our input vector into a higher dimension space. We can take our input vector and arrange it into a matrix $\mathbf{X} \in \mathbb{R}^{m \times n}$. We can then get a matrix $\Phi \in \mathbb{R}^{m \times p}$ from the following.

$$\Phi = \begin{bmatrix} \phi(\mathbf{x}_1) \\ \phi(\mathbf{x}_2) \\ \vdots \\ \phi(\mathbf{x}_m) \end{bmatrix} \quad (19)$$

And we can split our data into training and test matrices as well.

2.1.1 Gaussian Process Regression

We can have a naive formulation for this. The Gaussian Process Regression replaces the $\mathbf{x}$ with the nonlinear transformation as such:

$$f(\mathbf{x}) = \phi(\mathbf{x})^\top \mathbf{w} \quad (20)$$

, where $\mathbf{w} \in \mathbb{R}^p$. And for a case of training data, we have $\Phi \mathbf{w}$.

We have the posterior on $\mathbf{w}$ that we had previously but with different mean and precision matrix.

$$\mathbf{w} | \mathbf{X}, \mathbf{y} \sim N(\bar{\mathbf{w}}, \mathbf{A}^{-1}) \quad (21)$$

$$\bar{\mathbf{w}} = \sigma_m^{-2} \mathbf{A}^{-1} \Phi \mathbf{y} \quad (22)$$

$$\mathbf{A} = \sigma_m^{-2} \Phi^\top \Phi + \Sigma_p^{-1} \quad (23)$$

and we have a posterior on our predictions $\mathbf{f}^*$

$$\mathbf{f}^* | \mathbf{X}^*, \mathbf{X}, \mathbf{y} \sim N(\Phi^* \bar{\mathbf{w}}, \Phi^* \mathbf{A}^{-1} \Phi^{*\top}) \quad (24)$$

$$\bar{\mathbf{w}} = \sigma_m^{-2} \mathbf{A}^{-1} \Phi \mathbf{y} \quad (25)$$

$$\mathbf{A} = \sigma_m^{-2} \Phi^\top \Phi + \Sigma_p^{-1} \quad (26)$$

Since the matrix $\mathbf{A} \in \mathbb{R}^{p \times p}$, and $p \gg m$, then we have increased the computational complexity as we will have to compute the inverse of that matrix. Can we re-formulate the problem so we can address these computational complexity issues?

We can expand the formula for $\mathbf{f}^*$ to get an expanded version without the m, A variables. We make sure we only need to invert a matrix of size m so that we do not need to invert one of the much larger p .

2.1.2 Kernel Function

$\mathbf{K} \in \mathbb{R}^{m \times m}$ is the kernel matrix and is created by such:

$$\mathbf{K} = \Phi \Sigma_p \Phi^\top \quad (27)$$

where any general term is:

$$k_{ij} = k(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \Sigma_p \phi(\mathbf{x}_j) \quad (28)$$

. We can rearrange and this and split $\Sigma_p = \Sigma_p^{1/2} \Sigma_p^{1/2}$ to get

$$k_{ij} = \psi(\mathbf{x}_i)^\top \psi(\mathbf{x}_j) \quad (29)$$

where $\psi(\mathbf{x}) = \Sigma_p^{1/2} \phi(\mathbf{x})$. The kernel function then is the inner product of two training instances. The kernel trick seems to be: because the kernel function will output a scalar inner product matrix, which is symmetric and positive semi-definite, we do not actually need to compute the $\phi(\mathbf{x})$. We can instead just pick a kernel function that will output a positive semi-definite matrix. But how do we choose a kernel function?

The kernel functions tell us how instances are tied to each other. These kernel functions may have some properties:

- **Stationary:** $k(\mathbf{x}, \mathbf{x}')$ only depends on the difference $\mathbf{x} - \mathbf{x}'$.
- **Dot Product** $k(\mathbf{x}, \mathbf{x}')$ only depends on $\mathbf{x}^\top \mathbf{x}'$

We pick these properties if we have some information on the data and we want to preserve certain assumptions in the kernel

The squared exponential kernel is

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 e^{-\frac{1}{2}(\mathbf{x} - \mathbf{x}')^\top \mathbf{M}(\mathbf{x} - \mathbf{x}')} + \sigma_m^2 \mathbf{I}_m \quad (30)$$

where $\mathbf{M}$ is usually chosen to be $\frac{1}{l^2} \mathbf{I}_p$, where l is a parameter of length scale.

2.1.3 Function-Space View

The Gaussian Process Regression model allows for computing distributions over functions. We define a pdf for f :

$$f(\mathbf{x}) \sim GP(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')) \quad (31)$$

where m is the mean of f and k is the covariance.

The Gaussian Process defines a prior distribution on $f(\mathbf{x})$.